

MapReduce Example: World Temperature

(without combiners)

Mahmoud Parsian

Ph.D. in Computer Science

Where the Full Derivation Already Lives

This exact problem — average temperature per city — is already worked out step by step, key by key, in:

[mapreduce_examples/MapReduce_Find_Average_Temperature.md](#)

(Sections 1–9)

This deck instead walks through a **second, standalone worked example** with its own numbers — useful on its own, and as the baseline the companion deck,

[09_mapreduce_example_with_combiners.md](#),

extends with a combiner. Both should land on the same final answer.

The Problem

Input record: `<country>,<city>,<temperature>`

```
USA,Cupertino,73  
USA,Cupertino,73  
CANADA,Toronto,29  
CANADA,Toronto,48  
INDIA,Mumbai,68
```

Find the **average temperature per city** — and, from the same pass, the average temperature **per country** too.

The Mapper: One Record, Two Keys

Each input record contributes to *two* running averages at once — its city's, and its country's:

```
def map(key, value):  
    country, city, temperature = value.split(",")  
    if temperature >= 0:  
        # a mapper-side filter  
        emit(f"{country},{city}", temperature)  
        emit(country, temperature)
```

"USA,Cupertino,58" → emits both ("USA,Cupertino", 58) and ("USA", 58) — one record, two keys, computed in a single pass.

Worked Example: Input

USA,Cupertino,58	USA,Cupertino,78
USA,Cupertino,67	INDIA,Mumbai,90
USA,Sunnyvale,88	INDIA,Mumbai,96
USA,Sunnyvale,77	INDIA,Agra,98
	INDIA,Agra,92

Worked Example: Mapper Output

("USA,Cupertino", 58)	("USA", 58)
("USA,Cupertino", 67)	("USA", 67)
("USA,Sunnyvale", 88)	("USA", 88)
("USA,Sunnyvale", 77)	("USA", 77)
("USA,Cupertino", 78)	("USA", 78)
("INDIA,Mumbai", 90)	("INDIA", 90)
("INDIA,Mumbai", 96)	("INDIA", 96)
("INDIA,Agra", 98)	("INDIA", 98)
("INDIA,Agra", 92)	("INDIA", 92)

Worked Example: Sort & Shuffle Output

6 unique keys, (key, [values]) :

```
("USA,Cupertino", [58, 67, 78])
("USA,Sunnyvale", [88, 77])
("USA", [58, 67, 78, 88, 77])
("INDIA,Mumbai", [90, 96])
("INDIA,Agra", [98, 92])
("INDIA", [90, 96, 98, 92])
```

Worked Example: Reducer Output

```
def reduce(key, values):  
    emit(key, sum(values) / len(values))
```

```
("USA,Cupertino", 67.67)  
("USA,Sunnyvale", 82.5)  
("USA", 73.6)  
("INDIA,Mumbai", 93)  
("INDIA,Agra", 95)  
("INDIA", 94)
```


Try It Yourself

A third partition arrives with more records:

```
CANADA,Toronto,29  
CANADA,Toronto,48  
CANADA,Toronto,61
```

What does `reduce()` emit for `"CANADA,Toronto"` and for `"CANADA"` ?
(Same mapper/reducer as above — work it through, then check below.)

```
("CANADA,Toronto", (29+48+61)/3) -> 46.0  
("CANADA", (29+48+61)/3) -> 46.0  
# only one city so far, so equal
```

Next

- Filtering by temperature or by average — same rules as always:
[05_filters_in_mapreduce.md](#)
- The same base dataset (extended with a third partition, to show combiners working across multiple partitions), **redone with a combiner** — and why a naive combiner would silently break an average: [09_mapreduce_example_with_combiners.md](#)